Confluent Platform with Apache Flink Minikube Deployment

A Makefile-driven quickstart that deploys a full local streaming stack on Minikube:

- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.1** via the Confluent Flink Kubernetes Operator 1.130
- **Confluent Manager for Apache Flink (CMF) 2.1** for Flink environment management
- **Kafka UI** (Provectus) for cluster inspection

Table of Contents

- **1.0 Prerequisites**
- **2.0 Resource Requirements**
- **3.0 Architecture**
- **4.0 Quickstart**
  - **4.1 Full stack (CP + Kafka UI)**
  - **4.2 Add Apache Flink + CMF (run separately after `make cp-up`)**
- **5.0 Composite Workflow Reference**
- **6.0 Individual Target Reference**
  - **6.1 Phase 1 — Prerequisites**
  - **6.2 Phase 2 — Minikube**
  - **6.3 Phase 3 — Confluent Operator**
  - **6.4 Phase 4 — Confluent Platform**
  - **6.5 Phase 5 — Control Center**
  - **6.6 Phase 6 — Apache Flink**
  - **6.7 Phase 7 — Confluent Manager for Apache Flink (CMF)**
  - **6.8 Phase 8 — Kafka UI (Provectus)**
- **7.0 Configuration**
- **8.0 Repository Layout**
- **9.0 Teardown**
- **10.0 Manual Deployment Instructions**
- **11.0 Resources**

1.0 Prerequisites

macOS with Homebrew. To install all required tools in one step:

```
make install-prereqs
```

This installs `docker`, `kubectl`, `minikube`, `helm`, `envsubst`, and `gradle` via Homebrew. Once complete, **launch Docker Desktop** before proceeding.

To verify all tools are present without installing:

```
make check-prereqs
```

> Required: `docker`, `kubectl`, `minikube`, `helm`, `envsubst`, and `gradle`.

2.0 Resource Requirements

Minikube is configured with the following defaults, which are required to run the full stack:

Resource	Default
CPUs	6
Memory	20 GB
Disk	50 GB

Override any of these at the command line:

```
make cp-up MINIKUBE_CPUS=8 MINIKUBE_MEM=24576
```

3.0 Architecture

```
graph TD
    %% ── Composite entry points ──────────────────────────────
    CP_UP(["`**make cp-up**`"])
    FLINK_UP(["`**make flink-up**`"])
    CP_DOWN(["`**make cp-down**`"])
    FLINK_DOWN(["`**make flink-down**`"])
    TEARDOWN(["`**make confluent-teardown**`"])

    %% ── Phase 1: Prerequisites ──────────────────────────────
    subgraph P1["Phase 1 — Prerequisites"]
        CHECK_PRE["check-prereqs\ndocker · kubectl · minikube · helm"]
    end

    %% ── Phase 2: Minikube ───────────────────────────────────
    subgraph P2["Phase 2 — Minikube"]
        MK_START["minikube-start\ncpus=6 · mem=20GB · disk=50GB"]
        MK_STOP["minikube-stop"]
    end
```

```
    %% ── Phase 3: Confluent Operator
────────────────────────────────────────
    subgraph P3["Phase 3 — Confluent Operator"]
        NS["namespace\nkubectl create namespace confluent"]
        OP_INSTALL["operator-install\nhelm: confluent-for-kubernetes"]
        OP_UNINSTALL["operator-uninstall"]
    end

    %% ── Phase 4: Confluent Platform
────────────────────────────────────────
    subgraph P4["Phase 4 — Confluent Platform"]
        CP_DEPLOY["cp-deploy\nKafka KRaft · SR · Connect\nksqlDB · REST
Proxy · C3"]
        CP_DELETE["cp-delete"]
    end

    %% ── Phase 5: Control Center
──────────────────────────────────────────
    subgraph P5["Phase 5 — Control Center"]
        C3["c3-open\nlocalhost:9021"]
    end

    %% ── Phase 6: Apache Flink
───────────────────────────────────────────
    subgraph P6["Phase 6 — Apache Flink"]
        CERT["flink-cert-manager\ncert-manager v1.18.2"]
        FL_OP["flink-operator-install\nhelm: confluentinc/flink-
kubernetes-operator 1.130.0"]
        FL_DEPLOY["flink-deploy\nenvsubst → FlinkDeployment CR\ncp-
flink:2.1.1-cp1"]
        FL_UI["flink-ui\nlocalhost:8081"]
        FL_DELETE["flink-delete"]
        FL_OP_UN["flink-operator-uninstall"]
        CERT_UN["cert-manager-uninstall"]
    end

    %% ── Phase 7: CMF
─────────────────────────────────────────────────────
    subgraph P7["Phase 7 — Confluent Manager for Apache Flink (CMF)"]
        CMF_INSTALL["cmf-install\nhelm: confluent-manager-for-apache-flink
2.1.0"]
        CMF_ENV["cmf-env-create\nPOST /cmf/api/v1/environments"]
        CMF_OPEN["cmf-open\nlocalhost:8080/cmf/api/v1/environments"]
        CMF_PROXY["cmf-proxy-inject\nsocat sidecar → C3 Flink tab"]
        CMF_UN["cmf-uninstall"]
    end

    %% ── Phase 8: Kafka UI
──────────────────────────────────────────────
    subgraph P8["Phase 8 — Kafka UI"]
        KUI_INSTALL["kafka-ui-install\nhelm: provectus/kafka-
ui\nbootstrap: kafka:9071"]
        KUI_OPEN["kafka-ui-open\nllocalhost:8080"]
        KUI_UN["kafka-ui-uninstall"]
```

```
    end

    %% ── Manifests / Templates ──────────────────────────────────────────
    subgraph FS["k8s/base/"]
        MANIFEST[("flink-basic-deployment.yaml\nFLINK_IMAGE · FLINK_VERSION")]
    end

    %% ── make cp-up dependency chain ────────────────────────────────────
    CP_UP --> CHECK_PRE
    CP_UP --> MK_START
    CP_UP --> CP_CORE_UP
    CP_UP --> KUI_INSTALL

    CP_CORE_UP["cp-core-up"] --> OP_INSTALL
    CP_CORE_UP --> CP_DEPLOY
    OP_INSTALL --> NS

    %% ── make flink-up dependency chain ─────────────────────────────────
    FLINK_UP --> CERT
    FLINK_UP --> FL_OP
    FLINK_UP --> CMF_INSTALL
    FLINK_UP --> CMF_ENV
    FLINK_UP --> FL_DEPLOY
    FL_OP --> NS
    FL_DEPLOY --> MANIFEST

    %% ── make cp-down dependency chain ──────────────────────────────────
    CP_DOWN --> KUI_UN
    CP_DOWN --> CP_DELETE
    CP_DOWN --> OP_UNINSTALL

    %% ── make flink-down dependency chain ───────────────────────────────
    FLINK_DOWN --> FL_DELETE
    FLINK_DOWN --> CMF_UN
    FLINK_DOWN --> FL_OP_UN
    FLINK_DOWN --> CERT_UN

    %% ── make confluent-teardown ────────────────────────────────────────
    TEARDOWN -->|"1 — check minikube running"| FLINK_DOWN
    TEARDOWN -->|"2"| CP_DOWN
    TEARDOWN -->|"3 — delete namespace"| NS
    TEARDOWN -->|"4"| MK_STOP

    %% ── UI access ──────────────────────────────────────────────────────
    CP_DEPLOY -.->|"once Running"| C3
    FL_DEPLOY -.->|"once Running"| FL_UI
```

```
    CMF_INSTALL -.->|"once Running"| CMF_OPEN
    CMF_INSTALL -.->|"C3 Flink tab"| CMF_PROXY
    KUI_INSTALL -.->|"once Running"| KUI_OPEN

    %% ── Styles
────────────────────────────────────────────────────────────────
    classDef entry    fill:#1a1a2e,stroke:#e94560,color:#fff,font-
weight:bold
    classDef install  fill:#16213e,stroke:#0f3460,color:#a8dadc
    classDef remove   fill:#2d1b1b,stroke:#8b0000,color:#ffb3b3
    classDef ui       fill:#1b2d1b,stroke:#2d6a2d,color:#b3ffb3
    classDef file     fill:#2d2b1b,stroke:#8b7500,color:#ffe680
    classDef composite fill:#2a1a2e,stroke:#9b59b6,color:#dbb8ff

    class CP_UP,FLINK_UP,CP_DOWN,FLINK_DOWN,TEARDOWN entry
    class CP_CORE_UP composite
    class
CHECK_PRE,MK_START,NS,OP_INSTALL,CP_DEPLOY,CERT,FL_OP,FL_DEPLOY,CMF_INSTAL
L,CMF_ENV,CMF_PROXY,KUI_INSTALL install
    class
MK_STOP,OP_UNINSTALL,CP_DELETE,FL_DELETE,FL_OP_UN,CERT_UN,CMF_UN,KUI_UN
remove
    class C3,FL_UI,CMF_OPEN,KUI_OPEN ui
    class MANIFEST file
```

4.0 Quickstart

4.1 Full stack (CP + Kafka UI)

```
make cp-up
```

This runs: `check-prereqs` → `minikube-start` → `namespace` → `operator-install` → `cp-deploy` → `kafka-ui-install`.

Once pods are up, open Control Center:

```
make c3-open        # http://localhost:9021
```

4.2 Add Apache Flink + CMF (run separately after `make cp-up`)

```
make flink-up
```

This runs: `flink-cert-manager` → `flink-operator-install` → `cmf-install` → `cmf-env-create` → `flink-deploy`. `flink-up` is self-contained and can also be run standalone on a fresh cluster.

Once the Flink JobManager pod is running:

```
make flink-ui       # http://localhost:8081 (background — returns prompt)
make flink-ui-stop  # stop the background port-forward
make cmf-open       # http://localhost:8080/cmf/api/v1/environments
```

To expose the Flink tab inside Control Center, inject the CMF proxy sidecar:

```
make cmf-proxy-inject
```

5.0 Composite Workflow Reference

Target	What it does
`make cp-up`	Full stack: Minikube + CP + Kafka UI
`make flink-up`	cert-manager + Confluent Flink Operator + CMF + Flink cluster
`make cp-down`	Remove CP, Kafka UI, and Operator (Minikube keeps running)
`make flink-down`	Remove Flink cluster, CMF, Operator, and cert-manager
`make confluent-teardown`	Full teardown: everything + stop Minikube

6.0 Individual Target Reference

6.1 Phase 1 — Prerequisites

Target	Description
`install-prereqs`	Install Docker Desktop, kubectl, Minikube, Helm, envsubst, and Gradle via Homebrew
`check-prereqs`	Verify all required tools are available

6.2 Phase 2 — Minikube

Target	Description
`minikube-start`	Start Minikube with configured resources
`minikube-status`	Show Minikube and node status
`minikube-stop`	Stop the Minikube cluster
`minikube-delete`	Permanently delete the Minikube cluster

6.3 Phase 3 — Confluent Operator

Target	Description
`namespace`	Create the `confluent` namespace and set it as default context
`operator-install`	Add Confluent Helm repo and install CFK Operator
`operator-status`	Show CFK Operator pod status
`operator-uninstall`	Remove the CFK Operator Helm release

6.4 Phase 4 — Confluent Platform

Target	Description
`cp-deploy`	Deploy Kafka (KRaft), Schema Registry, Connect, ksqlDB, REST Proxy, Control Center
`cp-watch`	Watch pod startup live (Ctrl+C to exit)
`cp-status`	Show current pod status
`cp-delete`	Remove all CP components and leftover PVCs

6.5 Phase 5 — Control Center

Target	Description
`c3-open`	Port-forward Control Center in the background and open `http://localhost:9021` (`make c3-stop` to kill)
`c3-stop`	Stop the background Control Center port-forward

6.6 Phase 6 — Apache Flink

Target	Description
`flink-cert-manager`	Install cert-manager (Confluent Flink Operator dependency)
`flink-operator-install`	Install the Confluent Flink Kubernetes Operator (`confluentinc/flink-kubernetes-operator`)
`flink-operator-status`	Show Flink Operator pod status
`flink-operator-uninstall`	Remove the Confluent Flink Operator Helm release
`flink-deploy`	Deploy the Flink session cluster using `FLINK_MANIFEST`
`flink-status`	Show Flink pods and FlinkDeployment CRs
`flink-ui`	Port-forward Flink UI in the background and open `http://localhost:8081` (`make flink-ui-stop` to kill)

Target	Description
`flink-ui-stop`	Stop the background Flink UI port-forward
`flink-delete`	Delete the Flink session cluster
`cert-manager-uninstall`	Remove cert-manager

6.7 Phase 7 — Confluent Manager for Apache Flink (CMF)

Target	Description
`cmf-install`	Install CMF via Helm (`confluent-manager-for-apache-flink`) and wait for pod readiness
`cmf-env-create`	Create a Flink environment (`CMF_ENV_NAME`) in CMF pointing to the `confluent` namespace
`cmf-status`	Show CMF pod status and list registered Flink environments
`cmf-open`	Port-forward CMF REST API and open `http://localhost:8080/cmf/api/v1/environments`
`cmf-uninstall`	Uninstall CMF (safe to run even if not installed)
`cmf-proxy-inject`	Patch the C3 StatefulSet with a `socat` sidecar to expose the Flink tab in Control Center
`cmf-proxy-remove`	Remove the CMF proxy sidecar and resume CFK reconciliation
`cmf-proxy-logs`	Stream logs from the `cmf-proxy` sidecar in the C3 pod

6.8 Phase 8 — Kafka UI (Provectus)

Target	Description
`kafka-ui-install`	Install Kafka UI connected to the local CP cluster (Kafka + Schema Registry + Connect)
`kafka-ui-status`	Show Kafka UI pod status
`kafka-ui-open`	Port-forward Kafka UI and open `http://localhost:8080`
`kafka-ui-uninstall`	Remove Kafka UI

6.9 Phase 9 — Kafka Topics

Target	Description
`create-ptf-udf-topics`	Create the Kafka topics required by the ptf_udf Flink job

Target	Description
`delete-ptf-udf-topics`	Delete the Kafka topics used by the ptf_udf Flink job
`list-topics`	List all Kafka topics in the cluster
`produce-ptf-udf-sample`	Produce sample JSON records to the `user-events` topic
`consume-ptf-udf-output`	Consume records from the `enriched-events` topic (Ctrl+C to stop)

6.10 Phase 10 — Build & Deploy Flink JARs

Target	Description
`build-cp-java-ptf-udf`	Build the `ptf_udf` fat JAR (requires Gradle)
`deploy-cp-java-ptf-udf`	Build, create topics, upload to the Flink cluster, and submit the `ptf_udf` job

7.0 Configuration

All variables are overridable at the command line. Defaults:

Variable	Default	Description
`NAMESPACE`	`confluent`	Kubernetes namespace
`CONFLUENT_MANIFEST`	`k8s/base/confluent-platform-c3++.yaml`	Path to Confluent Platform manifest
`MINIKUBE_CPUS`	`6`	vCPUs allocated to Minikube
`MINIKUBE_MEM`	`20480`	Memory in MB
`MINIKUBE_DISK`	`50g`	Disk size
`FLINK_OPERATOR_VER`	`1.130.0`	Confluent Flink Kubernetes Operator version
`FLINK_IMAGE`	`confluentinc/cp-flink:2.1.1-cp1-java21-arm64`	Flink container image
`FLINK_VERSION`	`v2_1`	Flink API version string for the FlinkDeployment CR
`FLINK_CLUSTER_NAME`	`flink-basic`	Name of the FlinkDeployment resource
`FLINK_MANIFEST`	`k8s/base/flink-basic-deployment.yaml`	Path to FlinkDeployment template
`CERT_MANAGER_VER`	`v1.18.2`	cert-manager version
`CMF_VER`	`2.1.0`	Confluent Manager for Apache Flink version

Variable	Default	Description
`CMF_PORT`	`8080`	CMF REST API local port
`CMF_ENV_NAME`	`dev-local`	Flink environment name registered in CMF
`C3_PORT`	`9021`	Control Center local port
`FLINK_UI_PORT`	`8081`	Flink UI local port
`KAFKA_UI_PORT`	`8080`	Kafka UI local port
`PTF_UDF_TOPICS`	`user-events enriched-events`	Kafka topics for the ptf_udf Flink job

> **Note:** CMF uses the Confluent-packaged Flink operator (`confluentinc/flink-kubernetes-operator`) and `confluentinc/cp-flink` images — not the Apache OSS Flink operator or `flink` Docker Hub image.

Example — deploy a specific Flink image:

```
make flink-deploy FLINK_IMAGE=confluentinc/cp-flink:2.1.1-cp1-java21-arm64
FLINK_VERSION=v2_1
```

8.0 Repository Layout

```
.
├── Makefile
├── README.md
├── README.pdf
├── CHANGELOG.md
├── CHANGELOG.pdf
├── KNOWN_ISSUES.md
├── KNOWN_ISSUES.pdf
├── LICENSE.md
├── LICENSE.pdf
├── .gitignore
├── docs
│   ├── manual_deployment.md                # Step-by-step manual
deployment instructions (without Makefile)
│   └── manual_deployment.pdf
└── k8s/
    └── base/
        ├── confluent-platform-c3++.yaml    # Confluent Platform manifest
(KRaft + all components)
        └── flink-basic-deployment.yaml     # FlinkDeployment CR template
```

> The `flink-basic-deployment.yaml` is a template, `FLINK_IMAGE` and `FLINK_VERSION` are substituted at deploy time via `envsubst`. Do not apply it directly with `kubectl apply`.

9.0 Teardown

Remove everything and stop Minikube:

```
make confluent-teardown
```

To keep Minikube running but remove all deployed components:

```
make flink-down   # Flink cluster + CMF + operator + cert-manager
make cp-down      # CP + Kafka UI + CFK Operator
```

10.0 Manual Deployment Instructions

For users who want to understand the underlying steps without using the Makefile, see docs/manual_deployment.md.

11.0 Resources

- Manage Confluent Platform with Confluent for Kubernetes
- Get Started with Confluent Platform for Apache Flink